FACULTY OF COMPUTER APPLICATIONS

Marwadi University
Marwadi Chandarana Group

NAAC A+

**Faculty of Computer Applications**

# BCA Sem-1 (A)
# (AY-2026-27)

**Sub Code: 05BC3103**
**(Programming Practices: 1)**

# List of experiments with Advanced/Design/Open ended experiments

| | |
|---|---|
| **Roll No :** | 92600527076 |
| **Full Name :** | Harshad Tingre |

**GitHub Link |** https://github.com/lord-HRS/c--language-practical-s-7076.git

## Sub. Faculty Name:
## Prof. Vipulkumar Bambhaniya

**Write a C Program with proper comment and indentation with Output**

## 1) Student Performance Analysis System

**Problem Definition:**

Develop a menu-driven C application to analyze the academic performance of a group of students. The system should accept student information such as roll number, name, marks obtained in different subjects, and attendance percentage. Based on the available data, the application should calculate total marks, percentage, grade, pass/fail status, class average, highest and lowest performers, and other meaningful performance indicators.

The program should allow the user to add, display, search, and analyze student records. Students should decide the appropriate use of arrays, structures, functions, conditional statements, loops, and other C programming concepts.

**Minimum Requirements:**

- Accept details of multiple students.
- Calculate total and percentage.
- Assign grades based on suitable criteria.
- Identify pass/fail students.
- Find highest and lowest performers.
- Calculate average performance.
- Provide a menu-driven interface.

**Open-Ended Aspect:**

Students can independently decide the grading criteria, data representation, functions, searching method, sorting technique, and additional performance indicators.

## 2) Library Management System

**Problem Definition:**

Design and implement a C-based library management system that maintains information about books available in a library. The system should provide facilities for adding book records, displaying available books, searching for a particular book, issuing and returning books, and identifying currently available or issued books.

Each book may contain information such as book ID, title, author, category, price, and availability status. The student should determine an appropriate representation of book information and develop suitable functions for different operations.

**Minimum Requirements:**

- Add book information.
- Display all books.
- Search books by ID, title, or author.
- Issue a book.
- Return a book.
- Display available books.
- Display issued books.

**Open-Ended Aspect:**

Students may extend the system with fine calculation, sorting, file storage, multiple search

options, and issue-date information.

**Problem Definition:**

**3) Inventory Management and Billing System**

Develop a C program to manage the inventory of a small shop. The application should maintain product details such as product ID, name, category, quantity, unit price, and stock status. The system should allow the user to add products, update stock, search products, generate customer bills, and identify products whose stock has reached a predefined minimum level.

The billing component should allow the user to purchase multiple products and calculate the total bill based on quantity and price. Students should determine how the inventory and billing information will be represented and processed.

**Minimum Requirements:**

- Add and display products.
- Search for products.
- Update stock quantity.
- Generate a bill.
- Calculate total amount.
- Identify low-stock products.

**Open-Ended Aspect:**

Students may implement discounts, taxes, customer information, bill numbers, file handling, sorting, and sales reports.

**4) Student Attendance Management System**

**Problem Definition:**

Develop a C application for maintaining and analyzing student attendance. The system should accept attendance information for multiple students over a specified period and calculate individual attendance percentages.

The program should identify students who satisfy or fail to satisfy the required attendance criteria. It should also generate summary information such as class attendance percentage, students with maximum attendance, students with minimum attendance, and students requiring improvement.

**Minimum Requirements:**

- Store student attendance information.
- Calculate attendance percentage.
- Determine eligibility based on a specified threshold.
- Display students below the threshold.
- Generate attendance summary.

**Open-Ended Aspect:**

Students can decide whether to represent attendance using arrays, structures, or files and can introduce monthly/subject-wise attendance analysis.

**5) Employee Payroll Management System**

**Problem Definition:**

Develop a C-based payroll application for an organization. The system should maintain employee information and calculate the gross salary, deductions, and net salary of each employee.

The application should accept employee details such as employee ID, name, designation, basic salary, allowances, and deductions. Based on suitable salary rules, the program should generate individual salary details and organizational salary summaries.

**Minimum Requirements:**

- Store employee information.
- Calculate allowances.
- Calculate gross salary.

- Calculate deductions.
- Calculate net salary.
- Display employee salary details.
- Identify employees based on salary criteria.

**Open-Ended Aspect:**

Students may define suitable salary slabs, introduce different employee categories, generate salary reports, and implement file handling.

## 1. Student Performance Analysis System

**Algorithm / design:** Storerecords in a structure array, useamenu loop to run operations, and use functions/calculations to analyse the records. Compile with a C compiler such as GCC.

## C Program

```
#include <stdio.h>
#include <string.h>

#define MAX 50
#define SUBJECTS 3

typedef struct {
    int roll;
    char name[50];
    float marks[SUBJECTS], attendance;
} Student;

float total(Student s) {
    return s.marks[0] + s.marks[1] + s.marks[2];
}
float percentage(Student s) {
    return total(s) / SUBJECTS;
}
char grade(float p) {
    if (p >= 90) return 'A';
    if (p >= 75) return 'B';
    if (p >= 60) return 'C';
    if (p >= 40) return 'D';
    return 'F';
}
void show(Student s) {
    float p = percentage(s);
    printf("\\nRoll: %d | Name: %s\\n", s.roll, s.name);
    printf("Total: %.2f / 300 | Percentage: %.2f%%\\n", total(s), p);
    printf("Grade: %c | Result: %s | Attendance: %.1f%%\\n",
           grade(p), p >= 40 ? "PASS" : "FAIL", s.attendance);
}
int main(void) {
    Student s[MAX];
    int n = 0, choice, i, key, found;
    do {
        printf("\\n--- STUDENT PERFORMANCE SYSTEM ---\\n");
        printf("1.Add 2.Display 3.Search 4.Analyse 0.Exit\\nChoice: ");
        scanf("%d", &choice);
        if (choice == 1) {
            if (n == MAX) { printf("Record limit reached.\\n"); continue; }
            printf("Roll number: "); scanf("%d", &s[n].roll);
            printf("Name (one word): "); scanf("%49s", s[n].name);
            for (i = 0; i < SUBJECTS; i++) {
                do {
                    printf("Marks in subject %d (0-100): ", i + 1);
                    scanf("%f", &s[n].marks[i]);
                } while (s[n].marks[i] < 0 || s[n].marks[i] > 100);
            }
            printf("Attendance percentage: "); scanf("%f", &s[n].attendance);
            n++;
            printf("Student added successfully.\\n");
        } else if (choice == 2) {
            if (n == 0) printf("No records available.\\n");
            for (i = 0; i < n; i++) show(s[i]);
        } else if (choice == 3) {
            printf("Enter roll number to search: "); scanf("%d", &key);
            found = 0;
            for (i = 0; i < n; i++)
                if (s[i].roll == key) { show(s[i]); found = 1; }
            if (!found) printf("Student not found.\\n");
        } else if (choice == 4) {
            if (n == 0) { printf("Add students first.\\n"); continue; }
            int high = 0, low = 0, pass = 0;
            float sum = 0;
            for (i = 0; i < n; i++) {
                float p = percentage(s[i]);
                sum += p;
                if (p > percentage(s[high])) high = i;
                if (p < percentage(s[low])) low = i;
                if (p >= 40) pass++;
            }
            printf("Class average: %.2f%%\\n", sum / n);
            printf("Pass: %d | Fail: %d\\n", pass, n - pass);
            printf("Highest: %s (%.2f%%)\\n", s[high].name, percentage(s[high]));
            printf("Lowest: %s (%.2f%%)\\n", s[low].name, percentage(s[low]));
        } else if (choice != 0) printf("Invalid choice.\\n");
    } while (choice != 0);
    return 0;
}
```

## Sample Output (illustrative)

```
--- STUDENTPERFORMANCESYSTEM   ---
1.Add 2.Display   3.Search 4.Analyse   0.Exit
Choice: 1
Roll number: 101
```

```
Name (one word): Asha
Marks in subject 1 (0-100): 85
Marks in subject 2 (0-100): 90
Marks in subject 3 (0-100): 80
Attendance    percentage:    92
Student   added   successfully.
Choice: 1
Roll number: 102
Name (one word): Ravi
Marks in subject 1 (0-100): 35
Marks in subject 2 (0-100): 42
Marks in subject 3 (0-100): 38
Attendance    percentage:    75
Student   added   successfully.
Choice: 4
Class average: 61.67%
Pass: 1 | Fail: 1
Highest: Asha (85.00%)
Lowest: Ravi (38.33%)
Choice: 0
```

**Criteria used:** A ≥ 90%, B ≥ 75%, C ≥ 60%, D ≥ 40%, otherwise F. A student with percentage ≥ 40% is marked PASS.

## 2. Library Management System

**Algorithm / design:** Storerecords in a structure array, use a menu loop to run operations, and use functions/calculations to analyse the records. Compile with a C compiler such as GCC.

## C Program

```
#include <stdio.h>
#include <string.h>

#define MAX 100
typedef struct {
    int id, issued;
    char title[60], author[50], category[30];
    float price;
} Book;

int main(void) {
    Book b[MAX];
    int n = 0, choice, i, id, found;
    do {
        printf("\\n--- LIBRARY MANAGEMENT SYSTEM ---\\n");
        printf("1.Add 2.Display all 3.Search ID 4.Issue 5.Return\\n");
        printf("6.Available 7.Issued 0.Exit\\nChoice: ");
        scanf("%d", &choice);
        if (choice == 1) {
            if (n == MAX) { printf("Library storage full.\\n"); continue; }
            printf("Book ID: "); scanf("%d", &b[n].id);
            printf("Title (use_underscores): "); scanf("%59s", b[n].title);
            printf("Author (one_word): "); scanf("%49s", b[n].author);
            printf("Category: "); scanf("%29s", b[n].category);
            printf("Price: "); scanf("%f", &b[n].price);
            b[n].issued = 0; n++;
            printf("Book added.\\n");
        } else if (choice == 2 || choice == 6 || choice == 7) {
            found = 0;
            for (i = 0; i < n; i++) {
                if (choice == 6 && b[i].issued) continue;
                if (choice == 7 && !b[i].issued) continue;
                printf("ID:%d Title:%s Author:%s Category:%s Price:%.2f Status:%s\\n",
                       b[i].id, b[i].title, b[i].author, b[i].category,
                       b[i].price, b[i].issued ? "Issued" : "Available");
                found = 1;
            }
            if (!found) printf("No matching books.\\n");
        } else if (choice == 3 || choice == 4 || choice == 5) {
            printf("Enter book ID: "); scanf("%d", &id);
            found = 0;
            for (i = 0; i < n; i++) {
                if (b[i].id == id) {
                    found = 1;
                    if (choice == 3)
                        printf("%d %s by %s | %s | Rs.%.2f | %s\\n",
                               b[i].id, b[i].title, b[i].author, b[i].category,
                               b[i].price, b[i].issued ? "Issued" : "Available");
                    if (choice == 4) {
                        if (!b[i].issued) { b[i].issued = 1; printf("Book issued.\\n"); }
                        else printf("Book is already issued.\\n");
                    }
                    if (choice == 5) {
                        if (b[i].issued) { b[i].issued = 0; printf("Book returned.\\n"); }
                        else printf("Book was not issued.\\n");
                    }
                    break;
                }
            }
            if (!found) printf("Book ID not found.\\n");
        } else if (choice != 0) printf("Invalid choice.\\n");
    } while (choice != 0);
    return 0;
}
```

## Sample Output (illustrative)

```
--- LIBRARYMANAGEMENT SYSTEM ---
1.Add 2.Display all 3.Search ID 4.Issue 5.Return
6.Available 7.Issued 0.Exit
Choice: 1
Book ID: 11
Title (use_underscores): C_Programming
Author (one_word): Balagurusamy
Category: Education
Price: 450
Book added.
Choice: 4
Enter book ID: 11
Book issued.
Choice: 7
ID:11 Title:C_Programming Author:Balagurusamy Category:Education Price:450.00 Status:Issued
Choice: 5
Enter book ID: 11
```

```
Book returned.
Choice: 0
```

## 3. Inventory Management and Billing System

**Algorithm / design:** Storerecords in a structure array, useamenu loop to run operations, and use functions/calculations to analyse the records. Compile with a C compiler such as GCC.

## C Program

```
#include <stdio.h>
#include <string.h>

#define MAX 100
typedef struct {
    int id, qty;
    char name[50], category[30];
    float price;
} Product;

int main(void) {
    Product p[MAX];
    int n = 0, choice, i, id, qty, found;
    float bill;
    do {
        printf("\\n--- INVENTORY AND BILLING SYSTEM ---\\n");
        printf("1.Add product 2.Display 3.Search 4.Update stock\\n");
        printf("5.Generate bill 6.Low stock 0.Exit\\nChoice: ");
        scanf("%d", &choice);
        if (choice == 1) {
            if (n == MAX) { printf("Inventory full.\\n"); continue; }
            printf("Product ID: "); scanf("%d", &p[n].id);
            printf("Name (one_word): "); scanf("%49s", p[n].name);
            printf("Category: "); scanf("%29s", p[n].category);
            printf("Quantity: "); scanf("%d", &p[n].qty);
            printf("Unit price: "); scanf("%f", &p[n].price);
            n++; printf("Product added.\\n");
        } else if (choice == 2 || choice == 6) {
            found = 0;
            for (i = 0; i < n; i++) {
                if (choice == 6 && p[i].qty >= 5) continue;
                printf("ID:%d %s (%s) Qty:%d Price:Rs.%.2f\\n",
                       p[i].id, p[i].name, p[i].category, p[i].qty, p[i].price);
                found = 1;
            }
            if (!found) printf("No matching products.\\n");
        } else if (choice == 3 || choice == 4) {
            printf("Product ID: "); scanf("%d", &id); found = 0;
            for (i = 0; i < n; i++) if (p[i].id == id) {
                found = 1;
                if (choice == 3)
                    printf("%s | Qty:%d | Unit price:Rs.%.2f\\n",
                           p[i].name, p[i].qty, p[i].price);
                else {
                    printf("Quantity to add (positive number): ");
                    scanf("%d", &qty);
                    if (qty > 0) { p[i].qty += qty; printf("Stock updated.\\n"); }
                    else printf("Invalid quantity.\\n");
                }
                break;
            }
            if (!found) printf("Product not found.\\n");
        } else if (choice == 5) {
            bill = 0;
            printf("Enter product ID and quantity; enter ID 0 to finish.\\n");
            do {
                printf("Product ID: "); scanf("%d", &id);
                if (id == 0) break;
                printf("Quantity: "); scanf("%d", &qty); found = 0;
                for (i = 0; i < n; i++) if (p[i].id == id) {
                    found = 1;
                    if (qty > 0 && qty <= p[i].qty) {
                        float amount = qty * p[i].price;
                        bill += amount; p[i].qty -= qty;
                        printf("%s x %d = Rs.%.2f\\n", p[i].name, qty, amount);
                    } else printf("Invalid quantity or insufficient stock.\\n");
                    break;
                }
}               if (!found) printf("Product not found.\\n");
            } while (id != 0);
            printf("TOTAL BILL: Rs.%.2f\\n", bill);
        } else if (choice != 0) printf("Invalid choice.\\n");
    } while (choice != 0);
    return 0;
```

## Sample Output (illustrative)

```
--- INVENTORY AND BILLING SYSTEM ---
1.Add product 2.Display 3.Search 4.Update stock
5.Generate bill 6.Low stock 0.Exit
Choice: 1
Product ID: 201
Name (one_word): Notebook
Category: Stationery
Quantity: 20
```

```
Unit price: 50
Product added.
Choice: 5
Enter product ID and quantity; enter ID 0 to finish.
Product ID: 201
Quantity: 3
Notebook x 3 = Rs.150.00
Product ID: 0
TOTAL BILL: Rs.150.00
Choice: 6
ID:201  Notebook  (Stationery)  Qty:17  Price:Rs.50.00
Choice: 0
```

## 4. Student Attendance Management System

**Algorithm / design:** Storerecords in a structure array, useamenu loop to run operations, and use functions/calculations to analyse the records. Compile with a C compiler such as GCC.

## C Program

```
#include <stdio.h>

#define MAX 100
typedef struct {
    int roll, present, total;
    char name[50];
} Student;

float attendance(Student s) {
    if (s.total == 0) return 0;
    return (float)s.present * 100 / s.total;
}
int main(void) {
    Student s[MAX];
    int n = 0, choice, i, roll, found, high, low, eligible = 0;
    float sum;
    const float THRESHOLD = 75.0;
    do {
        printf("\\n--- ATTENDANCE MANAGEMENT SYSTEM ---\\n");
        printf("1.Add record 2.Display all 3.Search roll 4.Summary 5.Below threshold 0.Exit\\nChoice: ");
        scanf("%d", &choice);
        if (choice == 1) {
            if (n == MAX) { printf("Record limit reached.\\n"); continue; }
            printf("Roll number: "); scanf("%d", &s[n].roll);
            printf("Name (one word): "); scanf("%49s", s[n].name);
            printf("Total classes held: "); scanf("%d", &s[n].total);
            printf("Classes attended: "); scanf("%d", &s[n].present);
            if (s[n].total <= 0 || s[n].present < 0 || s[n].present > s[n].total) {
                printf("Invalid attendance values; record not added.\\n");
            } else { n++; printf("Attendance record added.\\n"); }
        } else if (choice == 2 || choice == 5) {
            found = 0;
            for (i = 0; i < n; i++) {
                if (choice == 5 && attendance(s[i]) >= THRESHOLD) continue;
                printf("Roll:%d Name:%s Attendance:%.2f%% Status:%s\\n",
                       s[i].roll, s[i].name, attendance(s[i]),
                       attendance(s[i]) >= THRESHOLD ? "Eligible" : "Below threshold");
                found = 1;
            }
            if (!found) printf("No matching records.\\n");
        } else if (choice == 3) {
            printf("Roll number: "); scanf("%d", &roll); found = 0;
            for (i = 0; i < n; i++) if (s[i].roll == roll) {
                printf("%s: %.2f%% attendance\\n", s[i].name, attendance(s[i]));
                found = 1; break;
            }
            if (!found) printf("Student not found.\\n");
        } else if (choice == 4) {
            if (n == 0) { printf("Add records first.\\n"); continue; }
            high = low = 0; sum = 0; eligible = 0;
            for (i = 0; i < n; i++) {
                sum += attendance(s[i]);
                if (attendance(s[i]) > attendance(s[high])) high = i;
                if (attendance(s[i]) < attendance(s[low])) low = i;
                if (attendance(s[i]) >= THRESHOLD) eligible++;
            }
            printf("Class average attendance: %.2f%%\\n", sum / n);
            printf("Eligible: %d | Below 75%%: %d\\n", eligible, n - eligible);
            printf("Maximum: %s (%.2f%%)\\n", s[high].name, attendance(s[high]));
            printf("Minimum: %s (%.2f%%)\\n", s[low].name, attendance(s[low]));
        } else if (choice != 0) printf("Invalid choice.\\n");
    } while (choice != 0);
    return 0;
}
```

## Sample Output (illustrative)

```
--- ATTENDANCE MANAGEMENT SYSTEM ---
1.Add record 2.Display all 3.Search roll 4.Summary 5.Below threshold 0.Exit
Choice: 1
Roll number: 101
Name (one word): Asha
Total classes held: 40
Classes attended: 36
Attendance record added.
Choice: 1
Roll number: 102
Name (one word): Ravi
Total classes held: 40
Classes attended: 25
Attendance record added.
Choice: 4
Class average attendance: 76.25%
Eligible: 1 | Below 75%: 1
```

```
Maximum: Asha (90.00%)
Minimum: Ravi (62.50%)
Choice: 5
Roll:102 Name:Ravi Attendance:62.50% Status:Below threshold
Choice: 0
```

**Eligibility rule:** Attendance of 75% or more is treated as eligible.

## 5. Employee Payroll Management System

**Algorithm / design:** Storerecords in a structure array, useamenu loop to run operations, and use functions/calculations to analyse the records. Compile with a C compiler such as GCC.

## C Program

```
#include <stdio.h>

#define MAX 100
typedef struct {
    int id;
    char name[50], designation[40];
    float basic, allowance, deduction, gross, net;
} Employee;

void calculate(Employee *e) {
    /* Allowance is 20% of basic salary; deduction is 8% of gross. */
    e->allowance = e->basic * 0.20f;
    e->gross = e->basic + e->allowance;
    e->deduction = e->gross * 0.08f;
    e->net = e->gross - e->deduction;
}
int main(void) {

    Employee e[MAX];
    int n = 0, choice, i, id, found;
    do {
        printf("\\n--- EMPLOYEE PAYROLL MANAGEMENT SYSTEM ---\\n");
        printf("1.Add employee 2.Display payroll 3.Search ID 4.High salary report 0.Exit\\nChoice: ");
        scanf("%d", &choice);
        if (choice == 1) {
            if (n == MAX) { printf("Employee limit reached.\\n"); continue; }
            printf("Employee ID: "); scanf("%d", &e[n].id);
            printf("Name (one word): "); scanf("%49s", e[n].name);
            printf("Designation (one_word): "); scanf("%39s", e[n].designation);
            printf("Basic salary: "); scanf("%f", &e[n].basic);
            if (e[n].basic < 0) { printf("Salary cannot be negative.\\n"); continue; }
            calculate(&e[n]);
            n++; printf("Employee added and salary calculated.\\n");

        } else if (choice == 2 || choice == 4) {
            found = 0;
            for (i = 0; i < n; i++) {
                if (choice == 4 && e[i].net <= 30000) continue;
                printf("\\nID:%d Name:%s Designation:%s\\n", e[i].id, e[i].name, e[i].designation);
                printf("Basic:Rs.%.2f Allowance:Rs.%.2f Gross:Rs.%.2f\\n",
                       e[i].basic, e[i].allowance, e[i].gross);
                printf("Deduction:Rs.%.2f Net salary:Rs.%.2f\\n", e[i].deduction, e[i].net);
                found = 1;
            }
            if (!found) printf("No matching employees.\\n");
        } else if (choice == 3) {
            printf("Employee ID: "); scanf("%d", &id); found = 0;
            for (i = 0; i < n; i++) if (e[i].id == id) {
                printf("%s | Gross:Rs.%.2f | Deduction:Rs.%.2f | Net:Rs.%.2f\\n",
                       e[i].name, e[i].gross, e[i].deduction, e[i].net);
                found = 1; break;
        }
        if (!found) printf("Employee not found.\\n");
}       } else if (choice != 0) printf("Invalid choice.\\n");
    } while (choice != 0);
    return 0;
```

## Sample Output (illustrative)

```
--- EMPLOYEE PAYROLL MANAGEMENT SYSTEM ---
1.Add employee 2.Display payroll 3.Search ID 4.High salary report 0.Exit
Choice: 1
Employee ID: 501
Name (one word): Asha
Designation (one_word): Developer
Basic salary: 40000
Employee added and salary calculated.
Choice: 2

ID:501 Name:Asha Designation:Developer
Basic:Rs.40000.00 Allowance:Rs.8000.00 Gross:Rs.48000.00
Deduction:Rs.3840.00 Net salary:Rs.44160.00
Choice: 4

ID:501 Name:Asha Designation:Developer
Basic:Rs.40000.00 Allowance:Rs.8000.00 Gross:Rs.48000.00
Deduction:Rs.3840.00 Net salary:Rs.44160.00
Choice: 0
```

**Payroll assumptions:** Allowance = 20% of basic salary; deduction = 8% of gross salary. These rates are sample rules and can be changed as instructed by faculty.